

PROJET PSMM

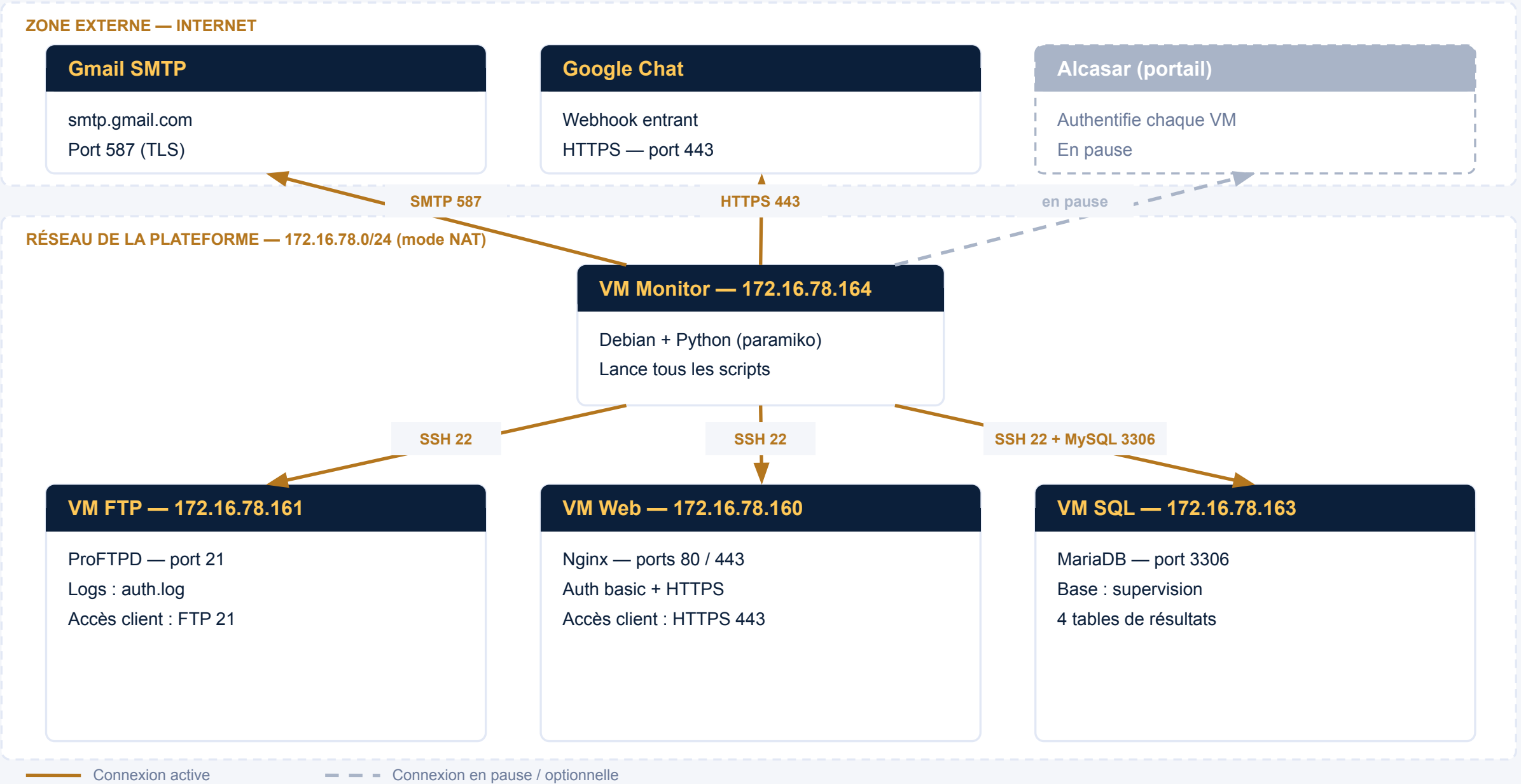
# Supervision de sécurité FTP, Web et SQL

# Les 4 machines utilisées

| Nom de la machine | Ce qu'elle fait                                              | Logiciel installé dessus |
|-------------------|--------------------------------------------------------------|--------------------------|
| Serveur FTP       | Permet d'envoyer des fichiers                                | ProFTPD                  |
| Serveur Web       | Affiche un site protégé par mot de passe                     | Nginx + HTTPS            |
| Serveur SQL       | Garde toutes les informations récoltées                      | MariaDB                  |
| VM Monitor        | Lance tous les scripts qui surveillent les 3 autres machines | Python                   |

Toutes les machines tournent sous Debian 13 et parlent entre elles en SSH.

# Architecture détaillée du projet PSMM



# Ce qu'on devait absolument respecter

1

## Clé SSH uniquement

Pas de mot de passe pour se connecter aux serveurs

2

## Compte dédié « monitor »

Le compte root ne peut plus se connecter à distance

3

## Site protégé

Mot de passe + connexion chiffrée (HTTPS)

4

## Mots de passe cachés

Rangés dans un fichier à part

5

## Sauvegardes régulières

Toutes les 3h, les 7 dernières gardées

6

## Alertes limitées

Pas plus d'un mail par heure pour ne pas spammer

## ÉTAPE 1

# Créer les comptes et les clés SSH

- Sur chaque machine, créer un compte appelé « monitor »
- Ce compte a le droit de faire des tâches d'admin (sudo)
- Créer une clé secrète sur la VM Monitor
- Copie de la partie publique de cette clé sur les 3 serveurs
- On vérifie qu'on peut se connecter sans taper de mot de passe

```
debian-monitor@debian-Monitor:~$ ssh -i ~/.ssh/id_ed25519_monitor monitor@172.16.78.161
Linux debian-ftp 6.12.107+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.107-1 (2026-08-29)
x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Oct  2 10:00:12 2026 from 172.16.78.164
monitor@debian-ftp:~$ exit
déconnexion
Connection to 172.16.78.161 closed.
debian-monitor@debian-Monitor:~$ █
```

FTP

```
ssh-keygen -t ed25519 \
-f ~/.ssh/id_ed25519_monitor
```

```
debian-monitor@debian-Monitor:~$ ssh -i ~/.ssh/id_ed25519_monitor monitor@172.16.78.163
Linux debian-sql 6.12.107+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.107-1 (2026-08-29)
x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Oct  1 19:05:08 2026 from 172.16.78.164
monitor@debian-sql:~$ exit
déconnexion
Connection to 172.16.78.163 closed.
```

SQL

```
debian-monitor@debian-Monitor:~$ ssh -i ~/.ssh/id_ed25519_monitor monitor@172.16.78.160
Linux debian-web 6.12.107+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.107-1 (2026-08-29)
x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Oct  2 10:00:37 2026 from 172.16.78.164
monitor@debian-web:~$ █
```

WEB

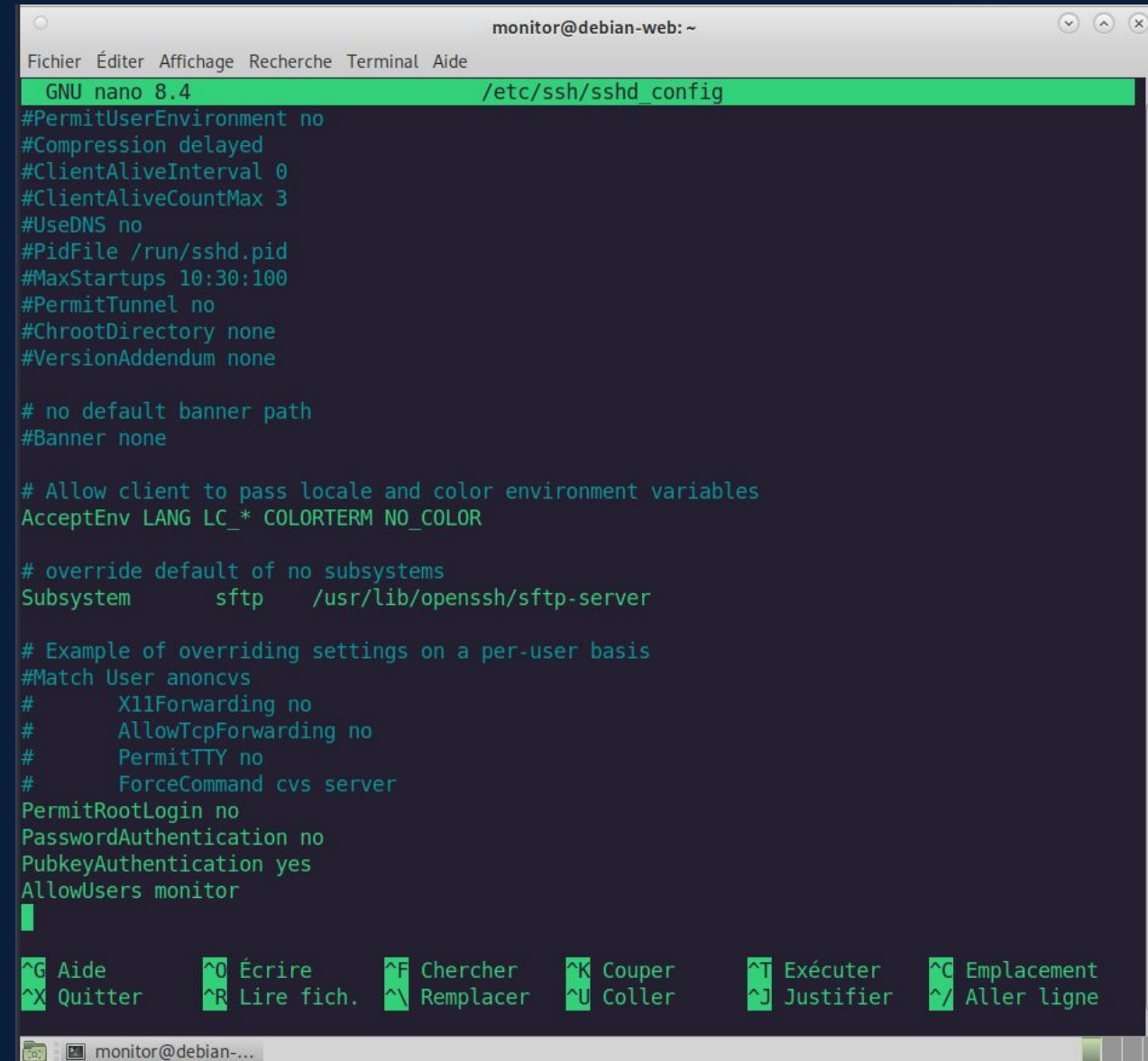
## ÉTAPE 2

# Fermer la porte au compte root

- Interdire au compte « root » de se connecter en SSH
- Interdire de se connecter avec un simple mot de passe
- Seule la clé secrète fonctionne, et seulement pour « monitor »

```
# /etc/ssh/sshd_config

PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers monitor
```



The screenshot shows a terminal window titled 'monitor@debian-web: ~' with a menu bar containing 'Fichier', 'Éditer', 'Affichage', 'Recherche', 'Terminal', and 'Aide'. The editor is GNU nano 8.4, editing /etc/ssh/sshd\_config. The file content is as follows:

```
GNU nano 8.4 /etc/ssh/sshd_config
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# Allow client to pass locale and color environment variables
AcceptEnv LANG LC_* COLORTERM NO_COLOR

# override default of no subsystems
Subsystem      sftp      /usr/lib/openssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
#      X11Forwarding no
#      AllowTcpForwarding no
#      PermitTTY no
#      ForceCommand cvs server
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers monitor
```

At the bottom of the terminal, there is a status bar with keyboard shortcuts: ^G Aide, ^O Écrire, ^F Chercher, ^K Couper, ^T Exécuter, ^C Emplacement, ^X Quitter, ^R Lire fich., ^\ Remplacer, ^U Coller, ^J Justifier, ^\_ Aller ligne.

### ÉTAPE 3

## Installer les logiciels sur chaque serveur

| Serveur | Commande utilisée                                 |
|---------|---------------------------------------------------|
| FTP     | <code>sudo apt install proftpd-basic</code>       |
| Web     | <code>sudo apt install nginx apache2-utils</code> |
| SQL     | <code>sudo apt install mariadb-server</code>      |

Chaque logiciel garde un historique (un « log ») de tout ce qui se passe : qui s'est connecté, à quelle heure, et si ça a marché. C'est dans ces historiques qu'on va aller chercher les tentatives ratées.

**FTP :** </var/log/proftpd/auth.log>

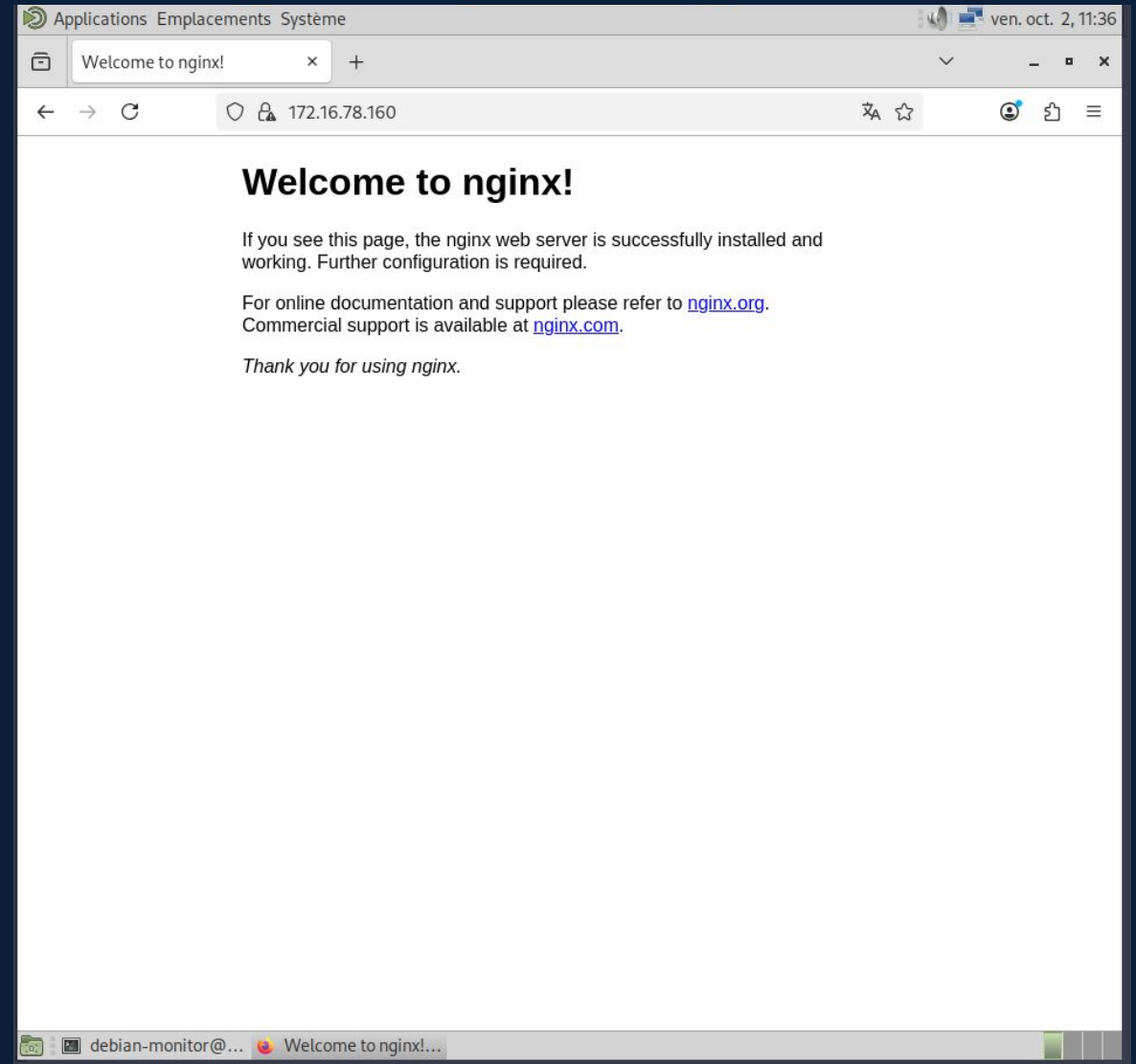
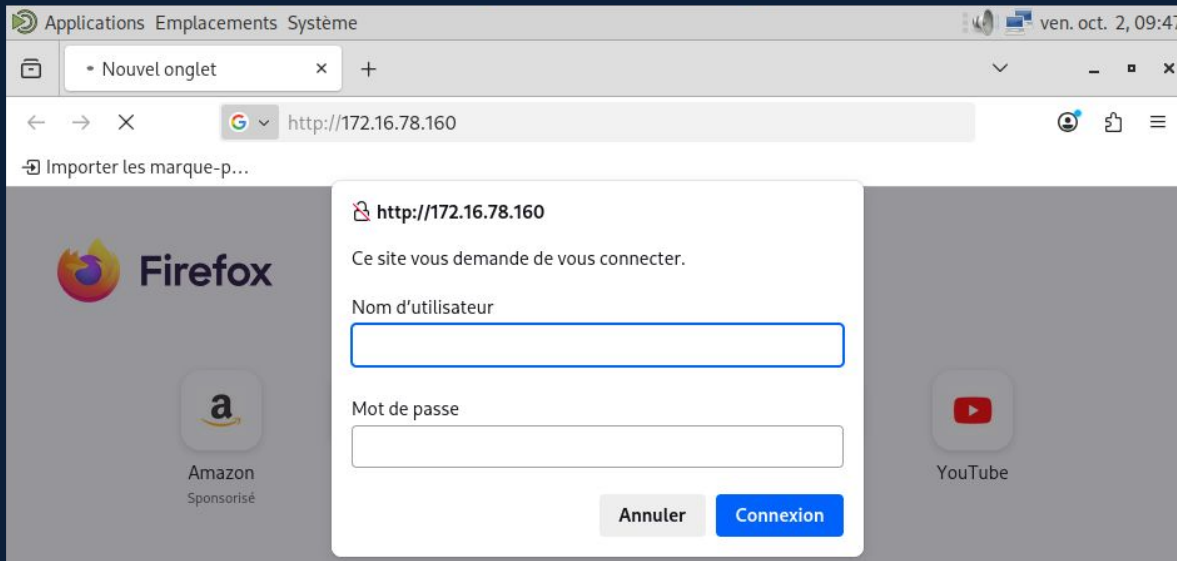
**Web :** </var/log/nginx/access.log>

**SQL :** </var/log/mysql/error.log>

## ÉTAPE 4

# Protéger le site web

- Le site demande un identifiant et un mot de passe
- La connexion passe en HTTPS (verrou, données chiffrées)



## ÉTAPE 5

# Créer la base de données

- Une base appelée « supervision » est créée sur le serveur SQL
- Elle contient 4 tableaux (tables) pour ranger les informations
- Un compte spécial permet aux scripts d'écrire dedans

```
CREATE TABLE sql_errors (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(100),  
    date_heure DATETIME,  
    ip_source VARCHAR(45)  
);
```

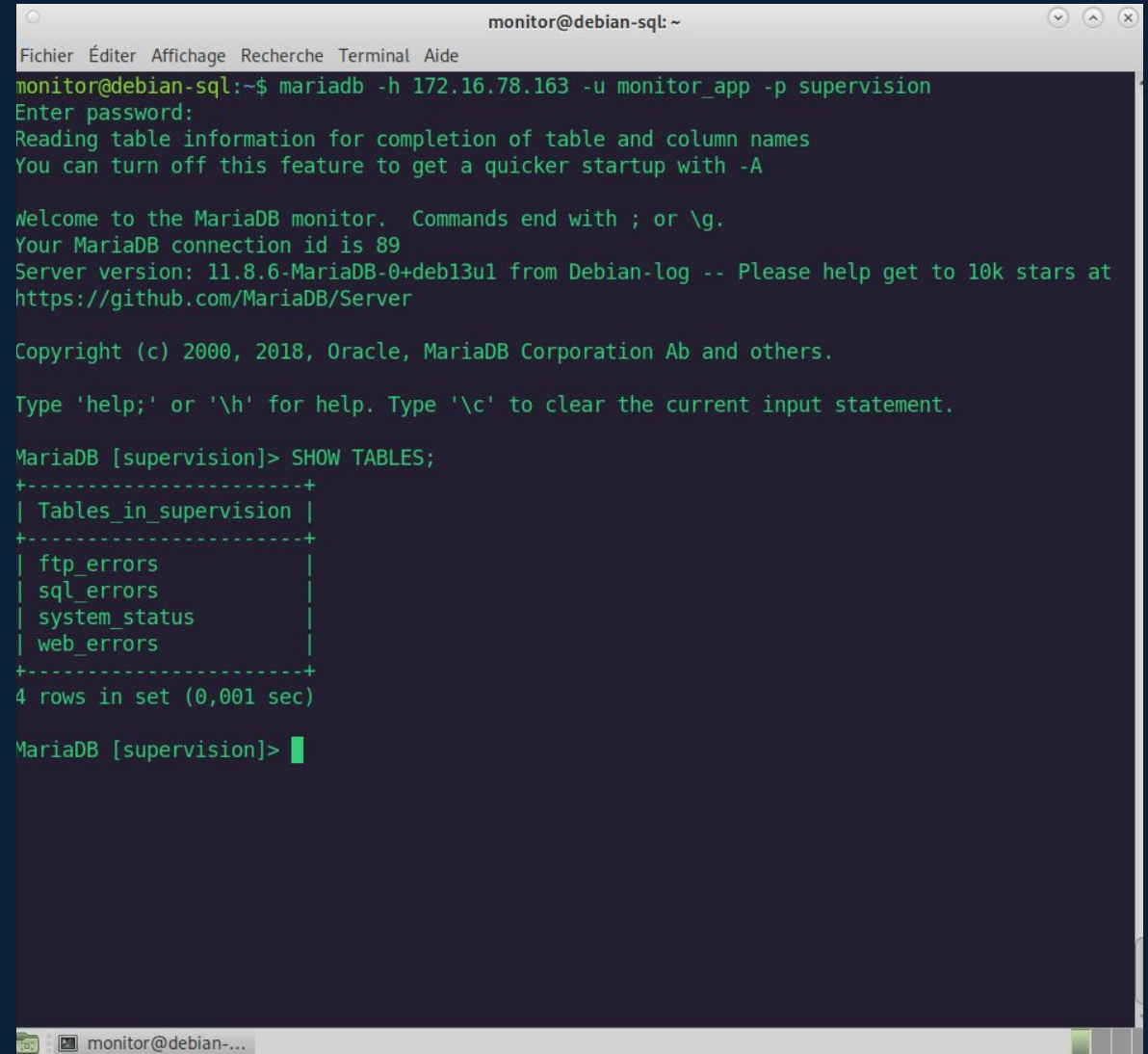
On crée un tableau appelé sql\_errors

Un numéro unique, qui s'incrémente tout seul à chaque ligne.

Le nom d'utilisateur tapé (texte, 100 caractères max).

La date et l'heure de la tentative.

L'adresse IP de celui qui a essayé de se connecter.



```
monitor@debian-sql: ~  
Fichier Éditer Affichage Recherche Terminal Aide  
monitor@debian-sql:~$ mariadb -h 172.16.78.163 -u monitor_app -p supervision  
Enter password:  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 89  
Server version: 11.8.6-MariaDB-0+deb13u1 from Debian-log -- Please help get to 10k stars at  
https://github.com/MariaDB/Server  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [supervision]> SHOW TABLES;  
+-----+  
| Tables_in_supervision |  
+-----+  
| ftp_errors            |  
| sql_errors            |  
| system_status         |  
| web_errors            |  
+-----+  
4 rows in set (0,001 sec)  
  
MariaDB [supervision]> █
```

ÉTAPE 6

# Les scripts qui vont chercher les erreurs

| Script             | Ce qu'il fait                                          |
|--------------------|--------------------------------------------------------|
| ssh_mysql_error.py | Va chercher les connexions refusées sur le serveur SQL |
| ssh_ftp_error.py   | Va chercher les connexions refusées sur le serveur FTP |
| ssh_web_error.py   | Va chercher les connexions refusées sur le site web    |

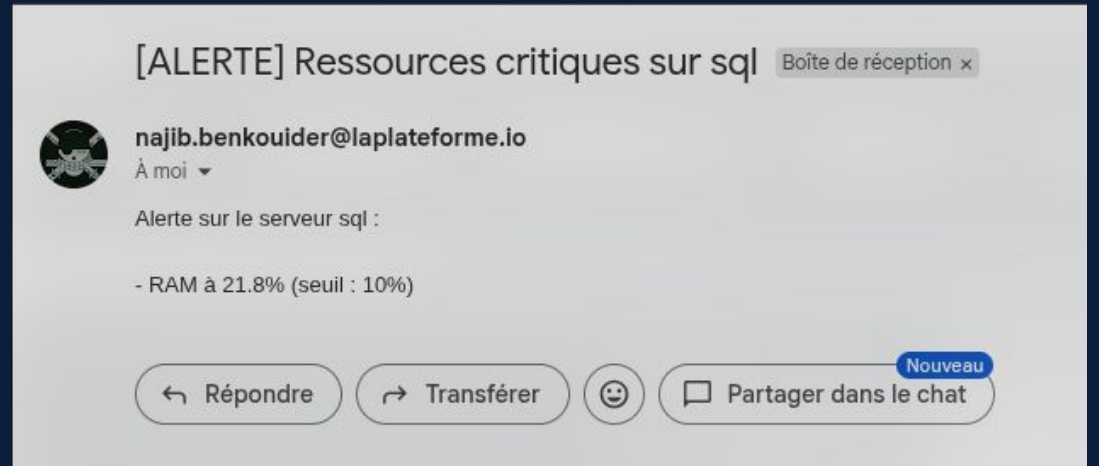
```

debian-monitor@debian-Monitor: ~
Fichier Éditer Affichage Recherche Terminal Aide
debian-monitor@debian-Monitor:~$ python3 ssh_mysql_error.py
8 tentative(s) SQL echouee(s) enregistree(s).
debian-monitor@debian-Monitor:~$ python3 ssh_ftp_error.py
4 tentative(s) FTP echouee(s) enregistree(s).
debian-monitor@debian-Monitor:~$ python3 ssh_web_error.py
3 tentative(s) web echouee(s) enregistree(s).
debian-monitor@debian-Monitor:~$
```

## ÉTAPE 7

# Le mail envoyé chaque jour

- Le script `ssh_serveur_mail.py` tourne chaque matin
- Il regarde toutes les tentatives ratées de la veille
- Il écrit un résumé et l'envoie par mail à l'administrateur



|                          |   |     |                                                                                                 |          |              |
|--------------------------|---|-----|-------------------------------------------------------------------------------------------------|----------|--------------|
| <input type="checkbox"/> | ☆ | moi | [ALERTE] Ressources critiques sur sql - Alerte sur le serveur sql : - RAM à 21.8% (seuil : 10%) | 22 sept. | <br><br><br> |
| <input type="checkbox"/> | ☆ | moi | [ALERTE] Ressources critiques sur web - Alerte sur le serveur web : - RAM à 31.4% (seuil : 10%) | 22 sept. |              |
| <input type="checkbox"/> | ☆ | moi | [ALERTE] Ressources critiques sur ftp - Alerte sur le serveur ftp : - RAM à 30.3% (seuil : 10%) | 22 sept. |              |
| <input type="checkbox"/> | ☆ | moi | [ALERTE] Ressources critiques sur sql - Alerte sur le serveur sql : - RAM à 21.6% (seuil : 10%) | 22 sept. |              |
| <input type="checkbox"/> | ☆ | moi | [ALERTE] Ressources critiques sur web - Alerte sur le serveur web : - RAM à 31.1% (seuil : 10%) | 22 sept. |              |
| <input type="checkbox"/> | ☆ | moi | [ALERTE] Ressources critiques sur ftp - Alerte sur le serveur ftp : - RAM à 30.3% (seuil : 10%) | 22 sept. |              |

## ÉTAPE 8

# Les sauvegardes automatiques

- Toutes les 3 heures, une copie de la base est faite
- Le nom du fichier contient la date et l'heure
- On garde seulement les 7 dernières copies, les plus vieilles sont supprimées

```
debian-monitor@debian-Monitor: ~/backups
Fichier Éditer Affichage Recherche Terminal Aide
debian-monitor@debian-Monitor:~/backups$ ls
supervision_20260923_150001.sql  supervision_20260930_120003.sql
supervision_20260924_120002.sql  supervision_20260930_150004.sql
supervision_20260929_145110.sql  supervision_20261001_180004.sql
supervision_20260929_150004.sql
debian-monitor@debian-Monitor:~/backups$
```

```
debian-monitor@debian-Monitor: ~
Fichier Éditer Affichage Recherche Terminal Aide
GNU nano 8.4 /tmp/crontab.wG0Bn3/crontab
0 10 * * * /usr/bin/python3 /home/debian-monitor/ssh_chat_notification.py >> /home/debian-m>
*/5 * * * * /usr/bin/python3 /home/debian-monitor/ssh_system_mail_v2.py >> /home/debian-mon>
0 */3 * * * /usr/bin/python3 /home/debian-monitor/ssh_cron_backup.py >> /home/debian-monito>
```

ÉTAPE 9

# Surveiller le CPU, la RAM et le disque

CPU

70 %

Au-dessus, une alerte part

RAM

80 %

Au-dessus, une alerte part

DISQUE

90 %

Au-dessus, une alerte part

[ALERTE] Ressources critiques sur sql

Boîte de réception x

 najib.benkouider@laplateforme.io

À moi ▼

Alerte sur le serveur sql :

- RAM à 21.8% (seuil : 10%)

Répondre

Transférer

😊

Partager dans le chat

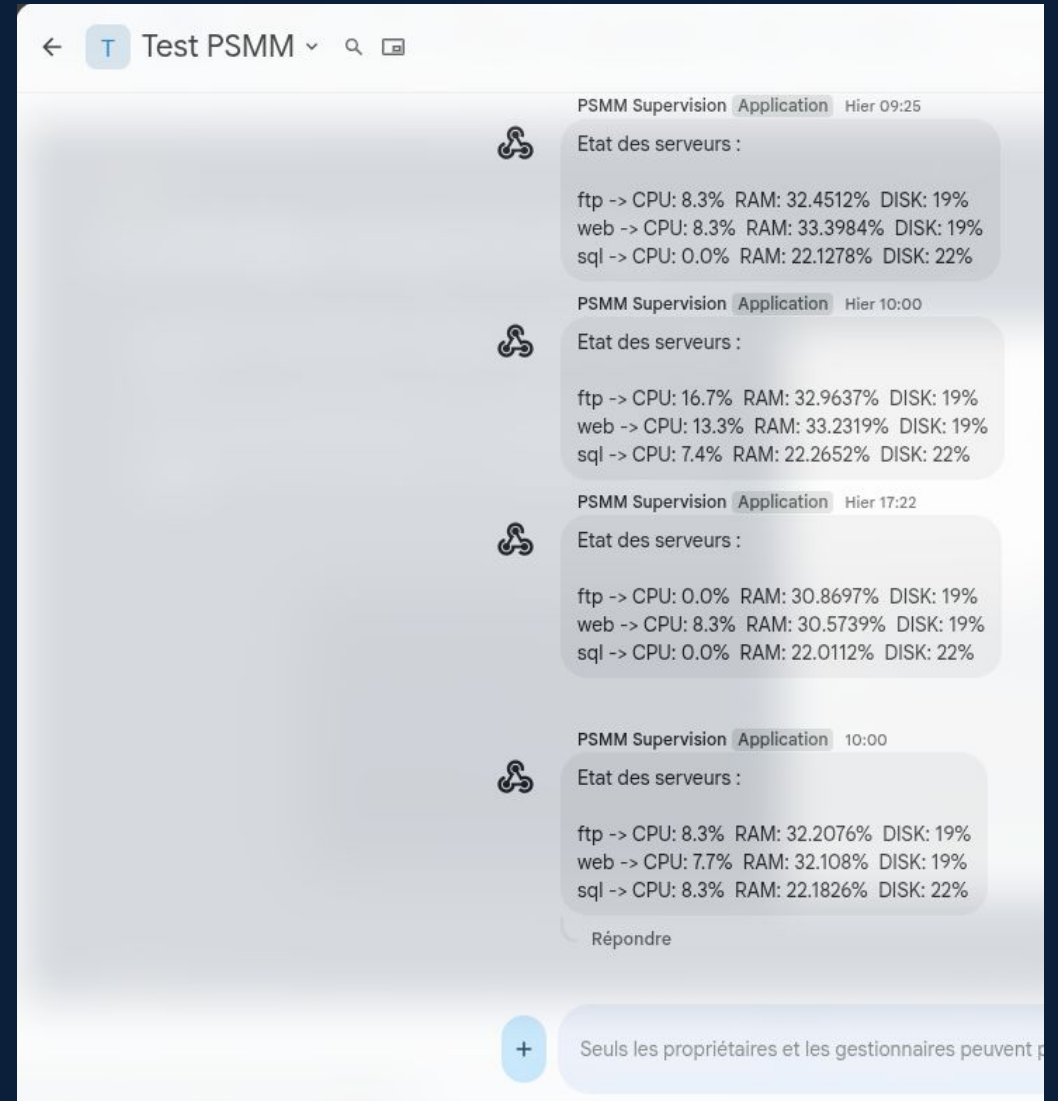
Nouveau

|                          |       |                                                                                                 |          |
|--------------------------|-------|-------------------------------------------------------------------------------------------------|----------|
| <input type="checkbox"/> | ☆ moi | [ALERTE] Ressources critiques sur sql - Alerte sur le serveur sql : - RAM à 21.8% (seuil : 10%) | 22 sept. |
| <input type="checkbox"/> | ☆ moi | [ALERTE] Ressources critiques sur web - Alerte sur le serveur web : - RAM à 31.4% (seuil : 10%) | 22 sept. |
| <input type="checkbox"/> | ☆ moi | [ALERTE] Ressources critiques sur ftp - Alerte sur le serveur ftp : - RAM à 30.3% (seuil : 10%) | 22 sept. |
| <input type="checkbox"/> | ☆ moi | [ALERTE] Ressources critiques sur sql - Alerte sur le serveur sql : - RAM à 21.6% (seuil : 10%) | 22 sept. |
| <input type="checkbox"/> | ☆ moi | [ALERTE] Ressources critiques sur web - Alerte sur le serveur web : - RAM à 31.1% (seuil : 10%) | 22 sept. |
| <input type="checkbox"/> | ☆ moi | [ALERTE] Ressources critiques sur ftp - Alerte sur le serveur ftp : - RAM à 30.3% (seuil : 10%) | 22 sept. |

## ÉTAPE 10

# Le message envoyé sur Google Chat

- Créer un espace de discussion sur Google Chat
- Un lien secret (webhook) permet au script d'y écrire
- Chaque matin à 10h, l'état des 3 serveurs est envoyé dans le chat



# Les tâches qui tournent toutes seules

| Ce qui se passe                 | Script                   | Quand                |
|---------------------------------|--------------------------|----------------------|
| Sauvegarde de la base           | ssh_cron_backup.py       | Toutes les 3 heures  |
| Vérification CPU / RAM / disque | ssh_system_mail_v2.py    | Toutes les 5 minutes |
| Message sur Google Chat         | ssh_chat_notification.py | Tous les jours à 10h |

```

    debian-monitor@debian-Monitor: ~
Fichier  Éditer  Affichage  Recherche  Terminal  Aide
GNU nano 8.4 /tmp/crontab.wG0Bn3/crontab
0 10 * * * /usr/bin/python3 /home/debian-monitor/ssh_chat_notification.py >> /home/debian-m
*/5 * * * * /usr/bin/python3 /home/debian-monitor/ssh_system_mail_v2.py >> /home/debian-mon
0 */3 * * * /usr/bin/python3 /home/debian-monitor/ssh_cron_backup.py >> /home/debian-monito
# Edit this file to introduce tasks to be run by cron.
"
```

## De la connexion SSH à la base de données

| Script                                                   | Ce qu'il fait                                              |
|----------------------------------------------------------|------------------------------------------------------------|
| ssh_login.py                                             | Se connecte à un serveur et lance une commande             |
| ssh_mysql.py                                             | Vérifie qu'on arrive bien à parler à la base               |
| ssh_mysql_error.py / ssh_ftp_error.py / ssh_web_error.py | Vont chercher les connexions ratées et les rangent en base |
| ssh_serveur_mail.py                                      | Envoie un résumé par mail chaque jour                      |
| ssh_cron_backup.py                                       | Fait une copie de sauvegarde de la base                    |
| ssh_system_mail_v2.py                                    | Surveille CPU/RAM/disque et prévient si besoin             |

## Les scripts python réalisé :

```
debian-monitor@debian-Monitor: ~  
Fichier Éditer Affichage Recherche Terminal Aide  
debian-monitor@debian-Monitor:~$ ls  
alcapass          dernier_mail_envoye.json  ssh_cron_backup.py      ssh_system_status.py  
alcasar           Documents                 ssh_ftp_error.py        ssh_update.py  
alcasar_out       Images                   ssh_login.py            ssh_web_error.py  
backup.log        Modèles                  ssh_login_sudo.py       ssh_web_error.py.save  
backups           Musique                   ssh_mysql_error.py      system_mail.log  
Bureau            Public                    ssh_mysql.py            Téléchargements  
chat.log          __pycache__              ssh_serveur_mail.py     Vidéos  
config_exemple.py secure_config.py          ssh_system_mail.py  
config.py         ssh_chat_notification.py  ssh_system_mail_v2.py  
debian-monitor@debian-Monitor:~$
```

# Exemples de script python :

```
debian-monitor@debian-Monitor: ~
Fichier Éditer Affichage Recherche Terminal Aide
GNU nano 8.4 ssh_ftp_error.py
import paramiko
import mysql.connector
import os
from config import *

# Etape 1 : se connecter en SSH au serveur FTP
cle = os.path.expanduser("~/ssh/id_ed25519_monitor")
ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh.connect("172.16.78.161", username="monitor", key_filename=cle)

# Etape 2 : lire le fichier de log
commande = "sudo cat /var/log/proftpd/auth.log"
entree, sortie, erreur = ssh.exec_command(commande, get_pty=True)
lignes = sortie.readlines()
ssh.close()

# Etape 3 : convertir les noms de mois francais en chiffres
mois = {
    "janv.": "01", "fevr.": "02", "mars": "03", "avr.": "04",
    "mai": "05", "juin": "06", "juil.": "07", "aout": "08",
    "sept.": "09", "oct.": "10", "nov.": "11", "dec.": "12"
}

# Etape 4 : parcourir chaque ligne du log pour trouver les echecs
liste_erreurs = []
utilisateur = ""
adresse_ip = ""
date_complete = ""

for ligne in lignes:
    [ Lecture de 75 lignes ]

^G Aide      ^O Écrire    ^F Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter    ^R Lire fich.^_ Remplacer  ^U Coller    ^J Justifier ^_ Aller ligne
debian-monitor@...
```

```
debian-monitor@debian-Monitor: ~
Fichier Éditer Affichage Recherche Terminal Aide
GNU nano 8.4 ssh_web_error.py *
import paramiko
import mysql.connector
import os
from datetime import datetime
from config import *

# Etape 1 : se connecter en SSH au serveur Web
cle = os.path.expanduser("~/ssh/id_ed25519_monitor")
ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh.connect("172.16.78.160", username="monitor", key_filename=cle)

# Etape 2 : lire le fichier de log
commande = "sudo cat /var/log/nginx/access.log"
entree, sortie, erreur = ssh.exec_command(commande, get_pty=True)
lignes = sortie.readlines()
ssh.close()

# Etape 3 : parcourir chaque ligne pour trouver les echecs(code 401)
liste_erreurs = []

for ligne in lignes:
    if " 401 " in ligne:
        adresse_ip = ligne.split(" ")[0]

        utilisateur = ligne.split(" - ")[1]
        utilisateur = utilisateur.split(" ")[0]

        morceau_date = ligne.split("[")[1]
        morceau_date = morceau_date.split("]")

^G Aide      ^O Écrire    ^F Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter    ^R Lire fich.^_ Remplacer  ^U Coller    ^J Justifier ^_ Aller ligne
debian-monitor@...
```

# Exemples de script python :

```
debian-monitor@debian-Monitor: ~
Fichier Éditer Affichage Recherche Terminal Aide
GNU nano 8.4 ssh_mysql_error.py
import paramiko
import mysql.connector
import os
from config import *

# Etape 1 : se connecter en SSH au serveur SQL
cle = os.path.expanduser("~/ssh/id_ed25519_monitor")
ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
ssh.connect("172.16.78.163", username="monitor", key_filename=cle)

# Etape 2 : lire le fichier de log
commande = "sudo cat /var/log/mysql/error.log"
entree, sortie, erreur = ssh.exec_command(commande, get_pty=True)
lignes = sortie.readlines()
ssh.close()

# Etape 3 : parcourir chaque ligne pour trouver les echecs de connexion
liste_erreurs = []
# Split les données
for ligne in lignes:
    if "Access denied for user" in ligne:
        date_complete = ligne.split(" ")[0] + " " + ligne.split(" ")[1]
        utilisateur = ligne.split("'")[1]
        adresse_ip = ligne.split("'")[3]

        liste_erreurs.append((utilisateur, date_complete, adresse_ip))

# Etape 4 : se connecter a la base de données
bdd = mysql.connector.connect(
    [ 52 lignes écrites ]
^G Aide      ^O Écrire    ^F Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^_ Remplacer  ^U Coller    ^J Justifier ^_ Aller ligne
debian-monitor@...
```

```
debian-monitor@debian-Monitor: ~
Fichier Éditer Affichage Recherche Terminal Aide
GNU nano 8.4 ssh_system_mail_v2.py
from datetime import datetime, timedelta
from config import *

# Seuils d'alerte (modifiables ici facilement)
SEUIL_CPU = 70
SEUIL_RAM = 80
SEUIL_DISK = 90

# Liste des serveurs a surveiller
serveurs = {
    "ftp": "172.16.78.161",
    "web": "172.16.78.160",
    "sql": "172.16.78.163"
}

cle = os.path.expanduser("~/ssh/id_ed25519_monitor")
fichier_suivi = os.path.expanduser("~/dernier_mail_envoye.json")

# Etape 1 : charger la date du dernier mail envoye pour chaque serveur
if os.path.exists(fichier_suivi):
    fichier = open(fichier_suivi, "r")
    derniers_envois = json.load(fichier)
    fichier.close()
else:
    derniers_envois = {}

# Etape 2 : se connecter a la base de donnees
bdd = mysql.connector.connect(
    host=DB_HOTE,
    user=DB_UTILISATEUR,
    password=DB_MOT_DE_PASSE,

```

# Ce que je n'ai pas testé jusqu'au bout

## Le script des mises à jour (ssh\_update.py)

- Le script est écrit en entier et fait tout ce qui est demandé
- Il doit se connecter à un portail internet (Alcasar) avant de pouvoir mettre à jour les serveurs
- Pour que ça marche, il fallait changer le réseau des machines (passer en « bridge »)
- Ce changement a rendu les adresses des machines instables, donc j'ai préféré revenir en arrière
- Le script reste prêt à être utilisé si on refait ce changement de réseau plus tard

# Exemples du script python **ssh\_update.py** :

```
debian-monitor@debian-Monitor: ~
Fichier Éditer Affichage Recherche Terminal Aide
GNU nano 8.4 ssh_update.py
import subprocess
import paramiko
import smtplib
import os
from email.mime.text import MIMEText
from config import *

cle = os.path.expanduser("~/ssh/id_ed25519_monitor")

serveurs = {
    "ftp": "172.16.78.161",
    "web": "172.16.78.160",
    "sql": "172.16.78.163"
}

# Etape 1 : lire les identifiants Alcasar dans le fichier .alcapass
fichier = open(os.path.expanduser("/home/debian-monitor/alcapass"), "r")
contenu = fichier.read()
fichier.close()

email_alcasar = contenu.split(" ")[0]
password_alcasar = contenu.split(" ")[1].strip()

# Etape 2 : se connecter a Alcasar (recuperation du token puis authentication)
commande_challenge = "curl -k -L --request GET https://alcasar.laplateforme.io/intercept.php"
resultat_challenge = subprocess.run(commande_challenge, shell=True, capture_output=True, text=True)
challenge = resultat_challenge.stdout.strip()

commande_login = [
    "curl", "--request", "POST",
    "--url", "https://alcasar.laplateforme.io/intercept.php",
    ]
```

```
debian-monitor@debian-Monitor: ~
Fichier Éditer Affichage Recherche Terminal Aide
GNU nano 8.4 ssh_update.py *
"--header", "Content-Type: multipart/form-data",
"--form", "username=" + email_alcasar,
"--form", "password=" + password_alcasar,
"--form", "challenge=" + challenge,
"--form", "button=Authentication",
"--location", "https://alcasar.laplateforme.io/intercept.php"
]
subprocess.run(commande_login, capture_output=True)

print("Connexion a Alcasar réussi.")

# Etape 3 : aller sur chaque serveur pour verifier et installer les mises a jour
liste_serveurs_a_redemarrer = []

for nom_serveur in serveurs:
    ip = serveurs[nom_serveur]

    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(ip, username="monitor", key_filename=cle)

    # on met a jour la liste des paquets disponibles
    ssh.exec_command("sudo apt update", get_pty=True)

    # on installe les mises a jour disponibles
    entree, sortie, erreur = ssh.exec_command("sudo apt upgrade -y", get_pty=True)
    resultat_upgrade = sortie.read().decode()

    print(nom_serveur + " : mise a jour effectuee.")

    # on verifie si un redemarrage est necessaire
```

## En entreprise, ça sert à :

- Repérer les tentatives de piratage (connexions avec mauvais mot de passe)
- Alerter l'administrateur rapidement
- Garder une trace, souvent obligatoire légalement
- Éviter les pannes grâce à la surveillance du serveur
- Pouvoir revenir en arrière grâce aux sauvegardes

# Technologies utilisées

Debian 13



Nginx



PHP 7.4 / 8.3



MariaDB



MySQL



ProFtpd



Python



OpenSSL



# Projet PSMM

[github.com/N3YGUIB/PSMM](https://github.com/N3YGUIB/PSMM)